

Tutorial ilustrado do laboratório

RAG customizada com Parent Document Retrieval, metadados, avaliação e chatbot Streamlit sobre o STPA Handbook

Versão: 08/06/2026 · Base: STPA_Handbook.pdf · Embeddings: text-embedding-3-small · Chat: gpt-4.1-mini (configurável)

Nota sobre modelos: gpt-4.1-mini ainda aparece disponível na API OpenAI, embora tenha sido retirado do ChatGPT. Mantenha OPENAI_CHAT_MODEL no .env e use gpt-5-mini ou gpt-4o-mini como fallback se necessário.



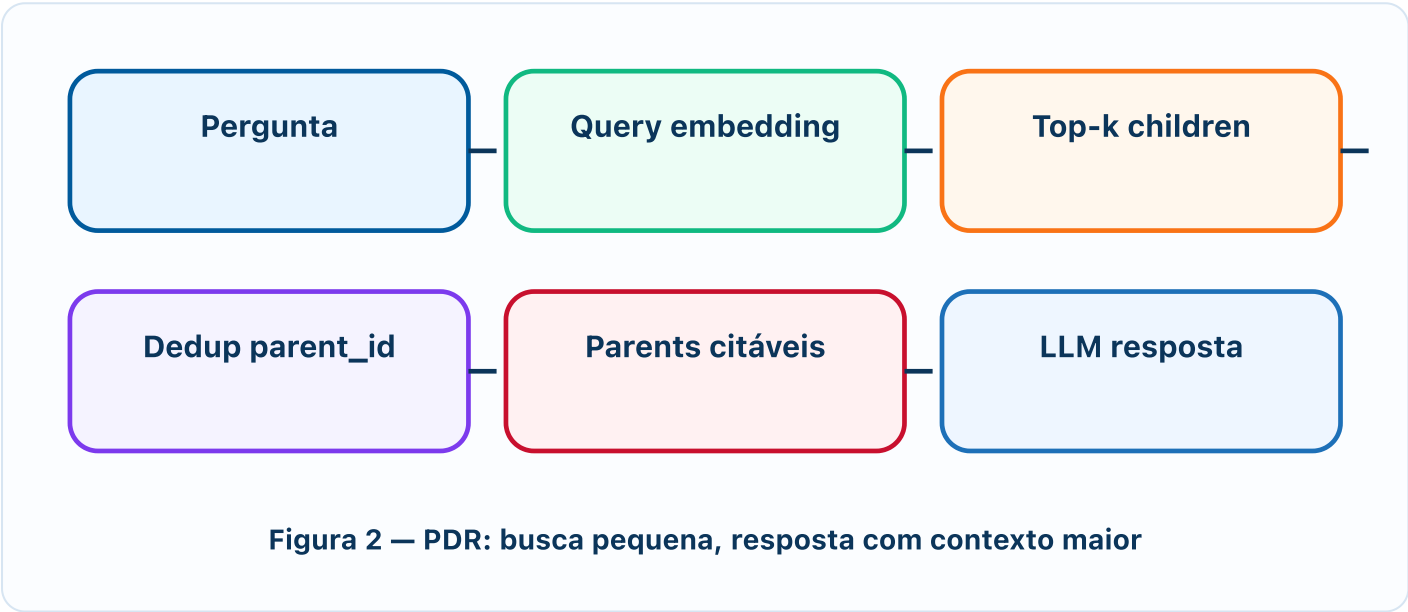
Figura 1 — Arquitetura de ponta a ponta

1. O que você vai construir

Você criará um chatbot web com Streamlit que responde perguntas factuais sobre STPA usando `STPA_Handbook.pdf` como base de conhecimento. O foco é aplicar as melhores práticas da Aula 11: parsing cuidadoso, PDR, metadados, citações, recusa quando não houver evidência e avaliação.

- Indexação: páginas → parents/children → embeddings OpenAI → ChromaDB local.
- Recuperação: busca nos child chunks, deduplicação por `parent_id`, resposta com parents citáveis.
- Interface: chat Streamlit com histórico, fontes recuperadas e filtros por `chunk_type`.

Regra de ouro: responda somente com contexto recuperado. Se a evidência não estiver no handbook, recuse explicitamente.



Aspecto	RAG ingênuo	RAG deste tutorial
Chunking	Janelas fixas	Parents + children com metadados
Citação	Opcional	Seção/páginas obrigatórias
Tabelas/figuras	Podem quebrar	Markdown/HTML para tabela; legenda/descrição para figura
Falha	Modelo inventa	Recusa ou re-query

2. Pré-requisitos multi-plataforma

Execute os comandos a partir de `Aula 11 – RAG Avançado` , salvo indicação contrária.

Sistema	Python	Terminal	Observações
Windows	Python 3.10+ (python.org)	PowerShell	Marque "Add Python to PATH". Use <code>py</code> se <code>python</code> falhar.
macOS	Python 3.10+ (python.org/Homebrew)	Terminal	Apple Silicon: prefira build arm64.
Linux	python3 + venv + pip	Bash/Zsh	Ubuntu/Debian: <code>sudo apt install python3 python3-venv python3-pip</code>

Verificações iniciais

```
# Windows
python --version

# macOS / Linux
python3 --version
```

Chave OpenAI

- Crie a chave no painel da OpenAI e configure limite de gastos.
- Guarde em `.env` ; nunca commite a chave.
- Embeddings: `text-embedding-3-small` . Chat: `gpt-4.1-mini` (ou fallback).

3. Estrutura do projeto

Windows PowerShell

```
cd "Aula 11 - RAG Avancado"
mkdir lab_stpa_rag_chatbot
cd lab_stpa_rag_chatbot
mkdir data, scripts, app, evals
mkdir data\raw, data\parsed, data\index
```

macOS / Linux

```
cd "Aula 11 - RAG Avancado"
mkdir -p lab_stpa_rag_chatbot/{data/raw,data/parsed,data/index,scripts,app,evals}
cd lab_stpa_rag_chatbot
```

Copiar o handbook

```
# Windows
Copy-Item "..\..\STPA_Handbook.pdf" "data\raw\STPA_Handbook.pdf"

# macOS / Linux
cp "../..\STPA_Handbook.pdf" "data/raw/STPA_Handbook.pdf"
```

4. Ambiente virtual e dependências

```
# Windows
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip

# macOS / Linux
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
```

Crie `requirements.txt`:

```
openai>=1.40.0
chromadb>=0.5.0
streamlit>=1.36.0
python-dotenv>=1.0.1
pypdf>=4.3.0
tiktoken>=0.7.0
rich>=13.7.1
```

```
pip install -r requirements.txt
```

Opcional para tabelas/layout: `pip install docling` ou `pip install "unstructured[pdf]"`. Inspecione sempre a extração antes de indexar.

5. Configuração segura (.env)

```
OPENAI_API_KEY=cole_sua_chave_aqui
OPENAI_CHAT_MODEL=gpt-4.1-mini
OPENAI_EMBED_MODEL=text-embedding-3-small
CHROMA_PATH=data/index/chroma
COLLECTION_NAME=stpa_handbook_children
MAX_PARENTS=3
```

`.gitignore` mínimo:

```
.env
.venv/
__pycache__/
data/index/chroma/
*.pyc
```

6. Script 01 — extrair páginas

Gera `data/parsed/pages.json` . Inspecione o texto antes de indexar.

```
# scripts/01_parse_pdf.py
from pathlib import Path
import json
from pypdf import PdfReader

PDF_PATH = Path("data/raw/STPA_Handbook.pdf")
OUT_PATH = Path("data/parsed/pages.json")

reader = PdfReader(str(PDF_PATH))
pages = []
for page_num, page in enumerate(reader.pages, start=1):
    text = page.extract_text() or ""
    pages.append({"page": page_num, "text": " ".join(text.split())})

OUT_PATH.parent.mkdir(parents=True, exist_ok=True)
OUT_PATH.write_text(json.dumps(pages, ensure_ascii=False, indent=2), encoding="utf-8")
print(f"Wrote {len(pages)} pages")
```

```
python scripts/01_parse_pdf.py
```

7. Script 02 — parents e children (PDR)



Figura 3 — Construção Parent/Child

Parte A — utilitários

```
# scripts/02_build_parent_child.py (trecho 1)
import json, re
from pathlib import Path

PAGES_PATH = Path("data/parsed/pages.json")
PARENTS_PATH = Path("data/parsed/parents.jsonl")
CHILDREN_PATH = Path("data/parsed/children.jsonl")
TARGET_CHILD_WORDS = 280
OVERLAP_WORDS = 45
PAGES_PER_PARENT = 2
```

Parte B — loop principal

```
pages = json.loads(PAGES_PATH.read_text(encoding="utf-8"))
parents, children = [], []

for i in range(0, len(pages), PAGES_PER_PARENT):
    group = pages[i:i + PAGES_PER_PARENT]
    text = "\n\n".join(p["text"] for p in group if p["text"].strip())
    if len(text.split()) < 80:
        continue
    page_start, page_end = group[0]["page"], group[-1]["page"]
    parent_id = f"stpa_p{page_start:03d}_{page_end:03d}"
    parent = {
        "id": parent_id,
        "text": text,
        "metadata": {
            "parent_id": parent_id,
            "section": "Unknown section",
            "page_start": page_start,
            "page_end": page_end,
            "source": "STPA_Handbook.pdf",
        },
    },
    parents.append(parent)
# dividir em children e salvar chunk_type='paragraph' ou 'table'
```

Para tabelas: salve Markdown/HTML em `text` e use `chunk_type='table'`. Para figuras: legenda + descrição conservadora.

8. Tabelas e imagens

Tipo	Representação	Observação
Tabela	Markdown/HTML	Preserva cabeçalhos e colunas
Figura	Legenda + descrição	Embedding texto não vê pixels
Diagrama	Vision-assisted ou ColPali	Use só se layout for crítico

```
{
  "id": "stpa_table_guidewords_c01",
  "text": "| Guideword | Meaning |\n|---|---|\n| Not provided | ... |",
  "metadata": {"chunk_type": "table", "section": "UCA", "page_start": 45}
}
```

9. Script 03 — embeddings e Chroma

```
# scripts/03_embed_index.py
import json, os
from pathlib import Path
import chromadb
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
chroma = chromadb.PersistentClient(path=os.getenv("CHROMA_PATH", "data/index/chroma"))
collection = chroma.get_or_create_collection(os.getenv("COLLECTION_NAME", "stpa_handbook_children"))

children = [json.loads(l) for l in Path("data/parsed/children.jsonl").read_text(encoding="utf-8").splitlines() if
l.strip()]
for i in range(0, len(children), 64):
    batch = children[i:i + 64]
    texts = [c["text"] for c in batch]
    resp = client.embeddings.create(model=os.getenv("OPENAI_EMBED_MODEL", "text-embedding-3-small"), input=texts)
    collection.upsert(
        ids=[c["id"] for c in batch],
        documents=texts,
        metadatas=[c["metadata"] for c in batch],
        embeddings=[d.embedding for d in resp.data],
    )
print(collection.count())
```

10. Núcleo RAG — app/rag_core.py

Parte A — carregar parents e embeddar query

```
import json, os
from pathlib import Path
import chromadb
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
EMBED_MODEL = os.getenv("OPENAI_EMBED_MODEL", "text-embedding-3-small")
CHAT_MODEL = os.getenv("OPENAI_CHAT_MODEL", "gpt-4.1-mini")
PARENTS = {json.loads(l)["id"]: json.loads(l) for l in Path("data/parsed/parents.jsonl").read_text(encoding="utf-8").splitlines() if l.strip()}

def embed_query(query: str) -> list[float]:
    return client.embeddings.create(model=EMBED_MODEL, input=query).data[0].embedding
```

Parte B — retrieve PDR

```
def retrieve_parents(query: str, n_results: int = 8, where=None):
    collection = chromadb.PersistentClient(path=os.getenv("CHROMA_PATH",
"data/index/chroma")).get_collection(os.getenv("COLLECTION_NAME", "stpa_handbook_children"))
    result = collection.query(query_embeddings=[embed_query(query)], n_results=n_results, where=where)
    hits, seen = [], set()
    for md, dist in zip(result["metadatas"][0], result["distances"][0]):
        pid = md["parent_id"]
        if pid in seen:
            continue
        seen.add(pid)
        parent = PARENTS.get(pid)
        if parent:
            hits.append({"parent_id": pid, "distance": dist, "text": parent["text"], "metadata": parent["metadata"]})
        if len(hits) >= int(os.getenv("MAX_PARENTS", "3")):
            break
    return hits
```

Parte C — gerar resposta com recusa

```
def answer_question(question: str, where=None):
    hits = retrieve_parents(question, where=where)
    context = "\n\n---\n\n".join(
        f"[Fonte {i}: {h['metadata'].get('section')} pp. {h['metadata'].get('page_start')}-"
        f"{h['metadata'].get('page_end')}] \n {h['text']}"
        for i, h in enumerate(hits, start=1)
    )
    system = ("Responda em português só com base no CONTEXTO. "
        "Se faltar evidência, diga: 'Informação não encontrada no STPA Handbook recuperado.' "
        "Inclua seção Referências.")
    resp = client.chat.completions.create(
        model=CHAT_MODEL, temperature=0,
        messages=[{"role": "system", "content": system}, {"role": "user", "content":
f"PERGUNTA:\n{question}\n\nCONTEXTO:\n{context}"},
        ]
    )
    return {"answer": resp.choices[0].message.content, "sources": hits}
```

11. Smoke test

```
# scripts/04_smoke_test_retriever.py
from app.rag_core import answer_question, retrieve_parents

q = "What is an unsafe control action in STPA?"
print(retrieve_parents(q))
print(answer_question(q)["answer"])
```

```
# macOS / Linux
PYTHONPATH=. python scripts/04_smoke_test_retriever.py

# Windows PowerShell
$env:PYTHONPATH="."
python scripts/04_smoke_test_retriever.py
```

12. Chatbot Streamlit

```
# app/streamlit_app.py
import streamlit as st
from rag_core import answer_question

st.set_page_config(page_title="STPA RAG Chatbot", layout="wide")
st.title("STPA Handbook · RAG Chatbot")

filter_mode = st.sidebar.selectbox("Filtro", ["Sem filtro", "Somente parágrafos", "Somente tabelas"])
where = {"chunk_type": "paragraph"} if filter_mode == "Somente parágrafos" else None
if filter_mode == "Somente tabelas":
    where = {"chunk_type": "table"}

if "messages" not in st.session_state:
    st.session_state.messages = [{"role": "assistant", "content": "Pergunte algo factual sobre STPA."}]

for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])

question = st.chat_input("Ex.: What is an unsafe control action?")
if question:
    st.session_state.messages.append({"role": "user", "content": question})
    result = answer_question(question, where=where)
    st.session_state.messages.append({"role": "assistant", "content": result["answer"]})
    with st.expander("Fontes recuperadas"):
        for i, src in enumerate(result["sources"], start=1):
            md = src["metadata"]
            st.markdown(f"**Fonte {i}** – {md.get('section')} · pp. {md.get('page_start')}-{md.get('page_end')}")
            st.text(src["text"][:1200])
```

```
# macOS / Linux
PYTHONPATH=app streamlit run app/streamlit_app.py

# Windows
$env:PYTHONPATH="app"
streamlit run app/streamlit_app.py
```

13. Avaliação

```
{ "id": "q01", "question": "What is the purpose of STPA?", "expected_contains": ["hazard", "control"] }
{ "id": "q02", "question": "What is an unsafe control action?", "expected_contains": ["control action"] }
{ "id": "q03", "question": "How are causal scenarios used in STPA?", "expected_contains": ["scenario"] }
```

Critério	Pergunta de auditoria	Sinal de problema
Factualidade	A resposta está no contexto?	Afirmações sem fonte
Citação	Há seção/página?	Referência ausente
Compleitude	Responde o pedido?	Resposta vaga
Recusa	Recusa sem evidência?	Invenção
Recuperação	Parents relevantes?	Seção errada

```
# evals/run_manual_eval.py
import json
from pathlib import Path
from app.rag_core import answer_question

for line in Path("evals/eval_questions.jsonl").read_text(encoding="utf-8").splitlines():
    item = json.loads(line)
    result = answer_question(item["question"])
    print(item["id"], result["answer"])
```

14. CRAG / Self-RAG (opcional)

Depois do baseline, adicione um avaliador que classifica a recuperação antes de gerar a resposta final.

```
def judge_retrieval(question, context):
    prompt = f'''Classifique a recuperação.
    Retorne apenas: correct, ambiguous ou incorrect.

    Pergunta: {question}
    Contexto: {context[:6000]}'''
    resp = client.chat.completions.create(
        model=CHAT_MODEL, temperature=0,
        messages=[{"role": "user", "content": prompt}],
    )
    return resp.choices[0].message.content.strip().lower()
```

Veredito	Ação
correct	Gerar resposta com os parents recuperados
ambiguous	Filtrar trechos ou buscar mais parents
incorrect	Reescrever query, trocar filtro ou recusar

Self-RAG simplificado: peça JSON com `retrieve_needed`, `passages_relevant` e `answer_supported`. Se `answer_supported=false`, não mostre a resposta.

15. Troubleshooting

Sintoma	Causa provável	Correção
<code>ModuleNotFoundError</code>	venv inativo	Ativar <code>.venv</code> e reinstalar deps
Erro 401 OpenAI	API key inválida	Revisar <code>.env</code>
Modelo indisponível	conta sem acesso	Trocar <code>OPENAI_CHAT_MODEL</code>
Chroma vazio	indexação não rodou	Executar <code>03_embed_index.py</code>
Resposta inventada	contexto fraco	temperature=0 + recusa explícita
Tabela ilegível	PDF linearizado	Docling + Markdown
Streamlit import error	PYTHONPATH	Usar <code>PYTHONPATH=app</code>

Checklist final

- PDF em `data/raw/STPA_Handbook.pdf`
- `pages.json`, `parents.jsonl`, `children.jsonl` gerados
- Coleção Chroma populada
- Smoke test retorna parents relevantes
- Chatbot cita páginas/seções e recusa sem evidência

16. Próximos passos

- Substituir parser baseline por Docling e comparar tabelas.
- Adicionar híbrido dense + BM25 e reranker.
- Implementar CRAG simplificado antes da resposta final.
- Criar relatório com 20 perguntas fixas e rubrica.
- Empacotar com Docker para reduzir diferenças entre SOs.



Figura 4 — Pipeline recomendado para entrega

Ao final, você terá um pipeline auditável de RAG customizada — não apenas um chatbot.

Apêndice operacional — versão detalhada para execução sem pesquisa externa

Este apêndice complementa o tutorial com todos os comandos, arquivos e verificações necessários para executar o laboratório de ponta a ponta. Use-o como roteiro operacional.

A. Estrutura completa esperada

```
lab_stpa_rag_chatbot/  
  data/raw/STPA_Handbook.pdf  
  data/parsed/pages.json  
  data/parsed/sample_pages.txt  
  data/parsed/parents.jsonl  
  data/parsed/children.jsonl  
  data/parsed/chunk_report.txt  
  data/index/chroma/  
  scripts/00_doctor.py  
  scripts/01_parse_pdf.py  
  scripts/02_build_parent_child.py  
  scripts/03_embed_index.py  
  scripts/04_smoke_test_retriever.py  
  app/rag_core.py  
  app/streamlit_app.py  
  evals/eval_questions.jsonl  
  evals/05_run_eval.py  
  .env  
  .gitignore  
  requirements.txt
```

B. Instalação e configuração

```
# 1. Ativar ambiente virtual antes de qualquer comando  
# Windows  
.\.venv\Scripts\Activate.ps1  
  
# macOS / Linux  
source .venv/bin/activate  
  
# 2. Criar requirements.txt com o conteúdo abaixo  
{requirements}  
  
# 3. Instalar  
pip install -r requirements.txt
```

C. Diagnóstico inicial — `scripts/00_doctor.py`

```
# scripts/00_doctor.py  
from __future__ import annotations  
  
import os  
import platform  
import sys  
from pathlib import Path  
  
from dotenv import load_dotenv  
  
REQUIRED_PATHS = [Path("data/raw/STPA_Handbook.pdf"), Path("requirements.txt"), Path(".env")]  
  
def check_import(name: str) -> bool:  
    try:  
        __import__(name)  
        return True  
    except Exception:  
        return False
```

```

def main() -> None:
    load_dotenv()
    print("Python:", sys.version)
    print("Platform:", platform.platform())
    print("Working directory:", Path.cwd())

    print("\nFiles:")
    for path in REQUIRED_PATHS:
        print(f" {'OK ' if path.exists() else 'MISS'} {path}")

    print("\nPackages:")
    for pkg in ["openai", "chromadb", "streamlit", "dotenv", "pypdf", "tiktoken"]:
        print(f" {'OK ' if check_import(pkg) else 'MISS'} {pkg}")

    print("\nEnvironment:")
    print(" OPENAI_API_KEY:", "OK" if os.getenv("OPENAI_API_KEY") else "MISSING")
    print(" OPENAI_CHAT_MODEL:", os.getenv("OPENAI_CHAT_MODEL", "not set"))
    print(" OPENAI_EMBED_MODEL:", os.getenv("OPENAI_EMBED_MODEL", "not set"))

    if not Path("data/raw/STPA_Handbook.pdf").exists():
        raise SystemExit("Copy STPA_Handbook.pdf to data/raw before continuing.")

if __name__ == "__main__":
    main()

```

Execute `python scripts/00_doctor.py`. Corrija qualquer item `MISS` antes de continuar.

D. Extração do PDF — `scripts/01_parse_pdf.py`

```

# scripts/01_parse_pdf.py
from __future__ import annotations

import json
from pathlib import Path

from pypdf import PdfReader

PDF_PATH = Path("data/raw/STPA_Handbook.pdf")
OUT_PATH = Path("data/parsed/pages.json")
SAMPLE_PATH = Path("data/parsed/sample_pages.txt")

def extract_page_text(page) -> str:
    try:
        text = page.extract_text(extraction_mode="layout") or ""
    except TypeError:
        text = page.extract_text() or ""
    return "\n".join(line.rstrip() for line in text.splitlines()).strip()

def main() -> None:
    if not PDF_PATH.exists():
        raise FileNotFoundError(f"Missing PDF: {PDF_PATH}")

    OUT_PATH.parent.mkdir(parents=True, exist_ok=True)
    reader = PdfReader(str(PDF_PATH))
    pages = []
    for page_num, page in enumerate(reader.pages, start=1):
        pages.append({"page": page_num, "text": extract_page_text(page)})

    OUT_PATH.write_text(json.dumps(pages, ensure_ascii=False, indent=2), encoding="utf-8")

    sample_numbers = [1, 2, 3, min(10, len(pages)), min(20, len(pages))]
    samples = []
    for n in sorted(set(sample_numbers)):
        if 1 <= n <= len(pages):
            samples.append(f"\n\n===== PAGE {n} =====\n{pages[n-1]['text'][:2500]}")
    SAMPLE_PATH.write_text("".join(samples), encoding="utf-8")

    empty = sum(1 for p in pages if not p["text"].strip())
    print(f"Wrote {len(pages)} pages to {OUT_PATH}")

```

```

print(f"Empty pages: {empty}")
print(f"Review sample text at {SAMPLE_PATH}")

if __name__ == "__main__":
    main()

```

Após executar, abra `data/parsed/sample_pages.txt`. Se as tabelas estiverem ilegíveis, documente a limitação ou use Docling/Unstructured para converter tabelas em Markdown.

E. Construção PDR — `scripts/02_build_parent_child.py`

```

# scripts/02_build_parent_child.py
from __future__ import annotations

import json
import re
from pathlib import Path

PAGES_PATH = Path("data/parsed/pages.json")
PARENTS_PATH = Path("data/parsed/parents.jsonl")
CHILDREN_PATH = Path("data/parsed/children.jsonl")
REPORT_PATH = Path("data/parsed/chunk_report.txt")

PAGES_PER_PARENT = 2
CHILD_WORDS = 280
OVERLAP_WORDS = 45
MIN_CHILD_WORDS = 80

SECTION_HINTS = [
    ("unsafe control action", "Unsafe Control Actions"),
    ("control structure", "Control Structure"),
    ("causal scenario", "Causal Scenarios"),
    ("loss", "Losses"),
    ("hazard", "Hazards"),
    ("constraint", "Safety Constraints"),
]

def load_pages() -> list[dict]:
    return json.loads(PAGES_PATH.read_text(encoding="utf-8"))

def write_jsonl(path: Path, rows: list[dict]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as f:
        for row in rows:
            f.write(json.dumps(row, ensure_ascii=False) + "\n")

def normalize_space(text: str) -> str:
    lines = [line.strip() for line in text.splitlines()]
    return "\n".join(line for line in lines if line)

def guess_section(text: str) -> str:
    first_lines = "\n".join(text.splitlines()[:12])
    match = re.search(r"\b(\d+(?:\.\d+){0,3})\s+[A-Z][A-Za-z0-9 ,\-\-()]{4,90})", first_lines)
    if match:
        return match.group(1).strip()
    lower = text.lower()
    for needle, label in SECTION_HINTS:
        if needle in lower:
            return label
    return "Unknown section"

def detect_chunk_type(text: str) -> str:
    pipe_lines = sum(1 for line in text.splitlines() if "|" in line)
    spaced_columns = sum(1 for line in text.splitlines() if re.search(r"\s{s{3,}}S", line))
    lower = text.lower()
    if pipe_lines >= 2 or spaced_columns >= 4:

```

```

        return "table"
    if "figure" in lower or "fig." in lower:
        return "figure_caption"
    return "paragraph"

def word_windows(text: str) -> list[str]:
    words = text.split()
    if len(words) <= CHILD_WORDS:
        return [text] if len(words) >= MIN_CHILD_WORDS else []
    chunks = []
    step = CHILD_WORDS - OVERLAP_WORDS
    for start in range(0, len(words), step):
        part = words[start:start + CHILD_WORDS]
        if len(part) >= MIN_CHILD_WORDS:
            chunks.append(" ".join(part))
    return chunks

def main() -> None:
    pages = load_pages()
    parents, children = [], []

    for i in range(0, len(pages), PAGES_PER_PARENT):
        group = pages[i:i + PAGES_PER_PARENT]
        text = normalize_space("\n\n".join(p["text"] for p in group if p["text"].strip()))
        if len(text.split()) < MIN_CHILD_WORDS:
            continue

        page_start = group[0]["page"]
        page_end = group[-1]["page"]
        section = guess_section(text)
        parent_id = f"stpa_p{page_start:03d}_{page_end:03d}"
        parent_md = {"parent_id": parent_id, "section": section, "page_start": page_start, "page_end":
page_end, "source": "STPA_Handbook.pdf"}
        parents.append({"id": parent_id, "text": text, "metadata": parent_md})

        for idx, child_text in enumerate(word_windows(text), start=1):
            child_id = f"{parent_id}_c{idx:02d}"
            children.append({"id": child_id, "text": child_text, "metadata": {**parent_md, "child_id":
child_id, "chunk_type": detect_chunk_type(child_text), "word_count": len(child_text.split())})

    write_jsonl(PARENTS_PATH, parents)
    write_jsonl(CHILDREN_PATH, children)
    by_type = {}
    for child in children:
        t = child["metadata"]["chunk_type"]
        by_type[t] = by_type.get(t, 0) + 1
    REPORT_PATH.write_text(f"parents={len(parents)}\nchildren={len(children)}\nchunk_types={by_type}\n",
encoding="utf-8")
    print(REPORT_PATH.read_text(encoding="utf-8"))

if __name__ == "__main__":
    main()

```

Verifique `chunk_report.txt` . Um resultado razoável deve ter dezenas de parents e centenas de children, dependendo do tamanho do PDF.

F. Indexação — `scripts/03_embed_index.py`

```

# scripts/03_embed_index.py
from __future__ import annotations

import json
import os
from pathlib import Path

import chromadb
from dotenv import load_dotenv
from openai import OpenAI

```

```

load_dotenv()
CHILDREN_PATH = Path("data/parsed/children.jsonl")
CHROMA_PATH = os.getenv("CHROMA_PATH", "data/index/chroma")
COLLECTION_NAME = os.getenv("COLLECTION_NAME", "stpa_handbook_children")
EMBED_MODEL = os.getenv("OPENAI_EMBED_MODEL", "text-embedding-3-small")
BATCH_SIZE = 64

def load_jsonl(path: Path) -> list[dict]:
    return [json.loads(line) for line in path.read_text(encoding="utf-8").splitlines() if line.strip()]

def batched(items: list, size: int):
    for i in range(0, len(items), size):
        yield items[i:i + size]

def main() -> None:
    if not os.getenv("OPENAI_API_KEY"):
        raise RuntimeError("OPENAI_API_KEY is missing. Check your .env file.")
    if not CHILDREN_PATH.exists():
        raise FileNotFoundError("Run scripts/02_build_parent_child.py first.")

    client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
    chroma = chromadb.PersistentClient(path=CHROMA_PATH)
    collection = chroma.get_or_create_collection(name=COLLECTION_NAME, metadata={"hnsw:space": "cosine"})
    children = load_jsonl(CHILDREN_PATH)
    print(f"Indexing {len(children)} children into {COLLECTION_NAME}")

    for batch in batched(children, BATCH_SIZE):
        texts = [item["text"].replace("\n", " ").strip() for item in batch]
        response = client.embeddings.create(model=EMBED_MODEL, input=texts)
        vectors = [item.embedding for item in sorted(response.data, key=lambda d: d.index)]
        collection.upsert(ids=[item["id"] for item in batch], documents=texts, metadatas=[item["metadata"] for
item in batch], embeddings=vectors)
        print(f" indexed {len(batch)}; total={collection.count()}")
    print("Done. Collection count:", collection.count())

if __name__ == "__main__":
    main()

```

Esse script deve terminar com `Collection count: N`, onde `N` é o número de children indexados.

G. Núcleo RAG — `app/rag_core.py`

```

# app/rag_core.py
from __future__ import annotations

import json
import os
from pathlib import Path
from typing import Any

import chromadb
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
EMBED_MODEL = os.getenv("OPENAI_EMBED_MODEL", "text-embedding-3-small")
CHAT_MODEL = os.getenv("OPENAI_CHAT_MODEL", "gpt-4.1-mini")
CHROMA_PATH = os.getenv("CHROMA_PATH", "data/index/chroma")
COLLECTION_NAME = os.getenv("COLLECTION_NAME", "stpa_handbook_children")
MAX_PARENTS = int(os.getenv("MAX_PARENTS", "3"))
PARENTS_PATH = Path("data/parsed/parents.jsonl")
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

def load_parents() -> dict[str, dict[str, Any]]:
    parents = {}
    for line in PARENTS_PATH.read_text(encoding="utf-8").splitlines():
        if line.strip():
            item = json.loads(line)

```

```

        parents[item["id"]] = item
    return parents

PARENTS = load_parents()

def get_collection():
    return chromadb.PersistentClient(path=CHROMA_PATH).get_collection(COLLECTION_NAME)

def embed_query(query: str) -> list[float]:
    response = client.embeddings.create(model=EMBED_MODEL, input=query.replace("\n", " ").strip())
    return response.data[0].embedding

def build_where_filter(chunk_type: str | None = None) -> dict | None:
    if chunk_type and chunk_type != "any":
        return {"chunk_type": chunk_type}
    return None

def retrieve_parents(question: str, n_results: int = 10, where: dict | None = None) -> list[dict[str, Any]]:
    result = get_collection().query(query=embed_query(question), n_results=n_results, where=where,
    include=["documents", "metadatas", "distances"])
    hits, seen = [], set()
    for metadata, distance, child_text in zip(result["metadatas"][0], result["distances"][0],
    result["documents"][0]):
        parent_id = metadata["parent_id"]
        if parent_id in seen:
            continue
        seen.add(parent_id)
        parent = PARENTS.get(parent_id)
        if parent:
            hits.append({"parent_id": parent_id, "distance": distance, "matched_child": child_text, "text":
parent["text"], "metadata": parent["metadata"]})
        if len(hits) >= MAX_PARENTS:
            break
    return hits

def format_context(hits: list[dict[str, Any]]) -> str:
    blocks = []
    for i, hit in enumerate(hits, start=1):
        md = hit["metadata"]
        ref = f"Fonte {i}: {md.get('source')} | seção: {md.get('section')} | páginas: {md.get('page_start')}-
{md.get('page_end')} | parent_id: {hit['parent_id']}"
        blocks.append(f"{{ref}}\n{{hit['text']}}")
    return "\n\n---\n\n".join(blocks)

def answer_question(question: str, chunk_type: str | None = None) -> dict[str, Any]:
    hits = retrieve_parents(question, where=build_where_filter(chunk_type))
    context = format_context(hits)
    system = ("Você é um assistente técnico para consulta ao STPA Handbook. Responda em português claro, com
precisão factual, usando somente o CONTEXTO. "
    "Se o contexto não contiver a resposta, diga exatamente: 'Informação não encontrada no STPA
Handbook recuperado.' "
    "Não use conhecimento externo. Sempre termine com uma seção 'Referências' listando fonte, seção e
páginas.")
    user = f"PERGUNTA:\n{question}\n\nCONTEXTO:\n{context}"
    response = client.chat.completions.create(model=CHAT_MODEL, temperature=0, messages=[{"role": "system",
"content": system}, {"role": "user", "content": user}])
    return {"answer": response.choices[0].message.content, "sources": hits}

```

H. Smoke test — scripts/04_smoke_test_retriever.py

```

# scripts/04_smoke_test_retriever.py
from app.rag_core import answer_question, retrieve_parents

QUESTION = "What is an unsafe control action in STPA?"
hits = retrieve_parents(QUESTION)
print(f"Retrieved parents: {len(hits)}")

```

```

for idx, hit in enumerate(hits, start=1):
    md = hit["metadata"]
    print(f"{idx}. {md.get('section')} | pp. {md.get('page_start')}-{md.get('page_end')} | distance={hit['distance']:.4f}")
print("\nAnswer:\n")
print(answer_question(QUESTION)["answer"])

```

```

# macOS / Linux
PYTHONPATH=. python scripts/04_smoke_test_retriever.py

# Windows PowerShell
$env:PYTHONPATH="."
python scripts/04_smoke_test_retriever.py

```

I. Chatbot — app/streamlit_app.py

```

# app/streamlit_app.py
from __future__ import annotations

import streamlit as st
from rag_core import answer_question

st.set_page_config(page_title="STPA RAG Chatbot", page_icon="📖", layout="wide")
st.title("STPA Handbook · RAG Chatbot")
st.caption("RAG customizada com PDR, ChromaDB e OpenAI API")

with st.sidebar:
    st.header("Configuração")
    filter_mode = st.selectbox("Filtro de conteúdo", ["Sem filtro", "Somente parágrafos", "Somente tabelas", "Somente legendas/figuras"])
    st.info("Use perguntas factuais. O chatbot deve citar páginas ou recusar.")

chunk_type = None
if filter_mode == "Somente parágrafos":
    chunk_type = "paragraph"
elif filter_mode == "Somente tabelas":
    chunk_type = "table"
elif filter_mode == "Somente legendas/figuras":
    chunk_type = "figure_caption"

if "messages" not in st.session_state:
    st.session_state.messages = [{"role": "assistant", "content": "Pergunte algo factual sobre STPA."}]

for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

question = st.chat_input("Ex.: What is an unsafe control action?")
if question:
    st.session_state.messages.append({"role": "user", "content": question})
    with st.chat_message("user"):
        st.markdown(question)
    with st.chat_message("assistant"):
        with st.status("Recuperando evidências...", expanded=False):
            result = answer_question(question, chunk_type=chunk_type)
            st.markdown(result["answer"])
            with st.expander("Fontes recuperadas"):
                for idx, source in enumerate(result["sources"], start=1):
                    md = source["metadata"]
                    st.markdown(f"**Fonte {idx}** - `{md.get('section')}` · pp. `{md.get('page_start')}-{md.get('page_end')}` · parent `{source.get('parent_id')}`")
                    st.text(source["text"][:1400] + ("..." if len(source["text"]) > 1400 else ""))
            st.session_state.messages.append({"role": "assistant", "content": result["answer"]})

```

```

# macOS / Linux
PYTHONPATH=app streamlit run app/streamlit_app.py

# Windows PowerShell

```



```
$env:PYTHONPATH="app"
streamlit run app/streamlit_app.py
```

J. Avaliação mínima

```
# evals/eval_questions.jsonl
{"id":"q01","question":"What is the purpose of STPA?","expected_contains":["hazard","control"]}
{"id":"q02","question":"What is an unsafe control action?","expected_contains":["control action"]}
{"id":"q03","question":"What is a control structure?","expected_contains":["controller","process"]}
{"id":"q04","question":"How are causal scenarios used in STPA?","expected_contains":["scenario"]}
```

```
# evals/05_run_eval.py
from __future__ import annotations
import json
from pathlib import Path
from app.rag_core import answer_question

for line in Path("evals/eval_questions.jsonl").read_text(encoding="utf-8").splitlines():
    item = json.loads(line)
    result = answer_question(item["question"])
    print("\n===", item["id"], item["question"])
    print(result["answer"])
    print("Sources:")
    for src in result["sources"]:
        md = src["metadata"]
        print("-", md.get("section"), md.get("page_start"), md.get("page_end"))
```

K. Checklist de validação

- `python scripts/00_doctor.py` sem itens críticos ausentes.
- `sample_pages.txt` inspecionado manualmente.
- `chunk_report.txt` mostra parents e children.
- `03_embed_index.py` termina com coleção não vazia.
- Smoke test recupera pelo menos 1 parent relevante.
- Streamlit abre e mostra fontes recuperadas.
- Pergunta fora do escopo gera recusa, não invenção.

L. Referências técnicas consultadas

- OpenAI API Models: <https://developers.openai.com/api/docs/models>
- OpenAI Embeddings: `text-embedding-3-small` para representação vetorial de texto.
- ChromaDB Docs: `query`, `get`, `where` e filtros por metadados.
- Streamlit Docs: `st.chat_message`, `st.chat_input` e `st.status`.
- pypdf Docs: `PdfReader` e `extract_text(extraction_mode="layout")`.